

Part 2 — PS Feature 1: AI Image Enhancer & Studio

The part of the system with the most measured evidence behind it, and one large admitted hole.

Q2.1 — "Background removal is a solved commodity. remove.bg has done it for years. What is AI about your image feature?"

Think. Concede the commodity immediately — arguing it is novel is how we lose. The defensible claim is about *which* model, chosen how, and about the four stages that are not matting.

Today — the concession. Correct. Matting is a commodity, and we say so in our own docs: *"Background removal is a commodity. This is not"* — where "this" is GeM category mapping (`web/api/channels/gem.py` docstring). Our image work is not novel research. It is a correctly assembled pipeline, benchmarked on our own photographs.

What is defensible — decision #1, closed 2026-08-28 against **five models over 41 fixtures** (research/segmentation/RESULTS.md):

Model	Licence	Peak VRAM	Median	Verdict
BiRefNet	MIT	1615 MiB	645 ms	chosen
BiRefNet-lite	MIT	851 MiB	273 ms	held in reserve
InSPyReNet	MIT	2924 MiB	~500 ms	close second
u2net	Apache-2.0	CPU	~450 ms	rejected — weakest on fringe
isnet-general-use	Apache-2.0	CPU	~950 ms	rejected — 48 holes per image
RMBG-2.0	CC BY-NC 4.0 —	never ran	non-commercial licence — could not have shipped	

Two things a judge should take from that table. **First, fringe.** Handicraft is tassels, jute fibre, jaali, filigree, loose thread. That is the whole category, and it is where a binary mask chops the pallu off. That was the open question and BiRefNet answered it. **Second, RMBG-2.0** — the model most recommended online for this task — is CC BY-NC. A good score would have changed nothing. We checked the licence before we checked the score.

Revisions are **pinned** (e2bf8e44...), because both repos ship `trust_remote_code` model definitions — the code that builds the network downloads with the weights. Tracking `main` means an upstream commit can silently move a result we put on a slide.

The four stages that are not matting and that actually decide whether a listing survives marketplace QC: white

balance, tone, denoise/sharpen, and **auto-crop to 85% fill with a programmatically asserted pure-white composite**. #FFFFFF exactly — marketplace systems flag (252,252,252) even when the eye cannot tell. We sample corner pixels and assert.

Breaks when. Adversarial fixtures: mean consensus IoU **0.616**, the worst group in the benchmark. Dark product on dark ground, transparent glass, specular brass. See Q2.4.

Q2.2 — "Your slide says the AI corrects lighting. Does it?"

Think. No. Say no in the first word. This is the one place where a hedge would be caught by reading our own repo, which a diligent judge could do if we publish it.

Answer. No — not yet, and the pipeline says so in every response.

CLAUDE.md, Current state, verbatim: *"Three stages inside that sequence are still unwritten and are skipped explicitly, with every response naming them: white_balance(), tone(), denoise_sharpen(). Colour is the significant absence."*

What runs end to end today (POST /enhance → 202 → poll GET /enhance/{job_id}): gate → 2000 px master → BiRefNet → tier → crop → per-target JPEG encode.

Why colour is the stage I would prioritise over everything else, and this is worth saying unprompted because it shows we know what matters: **for textiles, white balance matters more than background removal**. An artisan shoots under a tungsten bulb and the whole frame goes orange. A maroon Sambalpuri saree photographs orange. The buyer orders maroon, receives maroon, and it does not match the photograph — return, refund, and the artisan's rating drops. **Return rate destroys artisan income far faster than an unattractive background does.**

The design that is already decided and waiting for the implementation:

- **The white-paper reference trick.** The app asks *"product ke bagal me ek safed kagaz rakh dein."* We calibrate the exact white point from that paper and crop it out. Zero cost, professional-grade accuracy — better than a learned model and it costs one sentence of voice.
- **Blocked on one open request:** docs/Abhay/CHANGELOG.md records §9.2 — the white_ref rectangle on POST /enhance — as *"ours, unanswered"* from the app side. **Until the app sends that rectangle, white balance is gray-world forever.** That is a coordination gap between two people, not a research problem, and it is the single most valuable unblocked hour in the repo.
- **colour_confirmed gates publishing.** Non-negotiable rule 4 in CLAUDE.md: nothing publishes without it. White balance moves colour, and **only the person holding the object can say whether it is still true.** /capture/review asks by voice: *"kya yeh asli rang hai?"*

Breaks when. Right now, under a tungsten bulb, for every textile. The listing is still publishable and the artisan still confirms the colour by eye — but they are confirming an uncorrected photograph, so the confirmation is doing more work than it should.

Q2.3 — "You reject bad photos on the phone. How many good photos does your gate throw away?"

Think. They expect a hand-wave. We have the number, from 591 fixtures, and giving it immediately is worth more than the number being good.

Answer. 21 out of 93 — about 23% — after calibration. It was 65 out of 93 before.

Measured on **591 fixtures**: 93 photographs somebody chose to publish (the gate should pass these) and 498 degraded copies made by `images/degrade.py` with recorded parameters — 83 each of motion blur, defocus, under- and overexposure, off-centre and too-far. **The ground truth is a fact, not a judgement** (`research/RESULTS.md`, 2026-08-27).

	Original	Calibrated
Good photographs the capture gate refuses	65/93	21/93
Good photographs the server refuses	73/93	25/93
Degraded fixtures given the <i>right</i> spoken message	255/498	280/498
Degraded fixtures given the <i>wrong</i> message	200/498	79/498

The finding worth telling them, because it is the one nobody expects: `fill_fraction_max` was the single worst number in the project. At 0.90 it refused **44 of the 93 good photographs** as "too close" — including **9 of 9 patterned** fixtures and **15 of 18 fringe**. It caught nothing that needed catching. It is now **off**. A threshold that rejects half your good input and catches nothing is worse than no threshold, and we only found it because we labelled fixtures instead of eyeballing.

The second finding, which changed the architecture: the capture gate cannot see motion blur, because it measures a 240×180 downscale. A 24 px smear on a 1600×1600 fixture:

Measured on	Clean	24 px smear	Threshold = 100
240×180 — what <code>gate.js</code> sees	659	365	passes. Blur not detected
Full resolution — what the server sees	141	46	rejected, correctly

Downscaling $\sim 7\times$ turns a 24 px smear into ~ 3.5 px and the Laplacian recovers. So the capture blur check fails in **both** directions: it false-rejects sharp plain-weave fabric that has no edges to measure, and it false-accepts genuine hand-shake.

That is why blur is an advisory on the phone and a hard reject only on the server, at full resolution —

`blur_laplacian_variance_min = 100` advisory, `blur_laplacian_variance_reject_min = 20` server-side. Two numbers, deliberately not collapsed into one; there is a test asserting they have not been.

Both gates read the same file. `ai/thresholds.json`, fetched at runtime, never a second copy (`CLAUDE.md`). The `luma`, Laplacian and exposure functions live in one module — `ai/enhance/metrics.py` — imported by both the server gate and `images/check.py`, because they were briefly duplicated and *"a second copy that drifted by a rounding rule would mean the gate no longer does what the fixtures say it does."* Every threshold was calibrated with those exact functions.

Breaks when. 21 in 93 good photographs still get an unnecessary retake. And the fixture set is **web-sourced**, not shot by artisans on ₹7,000 phones in Indian light. That is the honest ceiling on all of these numbers, it is written in `research/RESULTS.md`, and a field fixture set is the next calibration round.

Q2.4 — "Show me the photograph your pipeline gets wrong."

Think. Have the list ready. Refusing to name a failure case reads as not having looked.

Today — five, from `docs/Master-Technical-Reference.md` §5.5, and the benchmark agrees:

Case	What goes wrong
Reflective metal — dhokra, bell metal, brass	Specular highlights blow out; segmentation confused
Transparent / translucent — glass, muslin, chanderi	Alpha matting; a binary mask slices the pallu off
Dark product on dark background	Segmentation fails outright
Fine fringes and tassels	Binary mask chops them — this is why BiRefNet won
Jewellery	Too small; phone macro struggles

Benchmark confirmation: **adversarial group mean consensus IoU 0.616**, against 0.99 for low contrast and 0.98 for a person-in-frame. And consensus IoU measures *agreement, not correctness* — five models can agree and all be wrong. We wrote that caveat into the results file rather than quoting 0.99 as accuracy.

What handles it today. A **tier system**. `apply_tier()` decides how much to trust the mask and degrades the output rather than producing a confidently wrong cutout. `CLAUDE.md` lists SAM 2 / EdgeSAM as **rejected for MVP** with the reason: *"the tier system covers the failure case."*

Rule 3 is the backstop: *"Losing the enhancement costs a prettier photo; it must never cost the artisan the listing."* A failed enhance falls back to the artisan's own photograph (`web/api/routers/products.py`), and the listing publishes.

At scale. SAM 2 tap-to-refine — the artisan taps the product when auto-matting fails — is the designed fallback and

is not built. For jewellery and glass specifically, the honest medium-term answer is a per-category route rather than one model for everything.

Breaks when. A dark brass idol on a dark cloth in low light. That photograph fails the gate, fails the matte, and has no white balance to save it. We would show the artisan their own unenhanced photo and publish it.

Q2.5 — "How fast is it, and what is the bottleneck?"

Think. Give the profile. The interesting part is that the bottleneck was never the model, and that finding is more impressive than a fast number.

Today. Measured 2026-09-02, RTX 4060, model warm, one 12 MP phone photograph:

Stage	Time	Where
JPEG decode	~100 ms	CPU
gate() blur/resolution	375 ms	CPU, full 12 MP array
to_master() 12 MP → 2000 px	168 ms	CPU LANCZOS
BiRefNet segment	402 ms	GPU
apply_tier()	124 ms	CPU
crop()	54 ms	CPU
encode ×3 targets	80 ms	CPU
Total compute	≈1.3 s	all local

Against that, object storage was costing ~7 seconds — eight serialized round trips at 0.81–1.13 s each, because the Supabase project sits in ap-southeast-2 (Sydney).

≈7 s of network against 1.3 s of work. Compute was 15% of the wall clock. The image crossed the Indian Ocean four times to be processed by a GPU in the same room.

Three multipliers, and **none of them is the model**: geography (Sydney is ~0.9 s RTT from India), serialization (seven independent puts running one after another), and round-tripping (`web/api` uploads bytes to Sydney so `ai/` can download the same bytes straight back).

Fixes, in order of size (`docs/app/Future-Implementations.md` §4):

Change	Effect
Move the project to <code>ap-south-1</code> (Mumbai)	0.9 s → ~50 ms per round trip. A project setting, not code
Parallelize the seven puts (<code>asyncio.gather</code>)	7 serial → ~1 concurrent, even from Sydney
<code>ai/</code> writes to object storage directly	Kills the laptop → cloud → laptop → cloud trip entirely. The real fix
Redis + RQ for the job table	Survives restart and more than one replica
CDN in front of the public bucket	Marketplace image loads, not processing

Two things the profile tells us not to do, and we wrote them down so a future optimiser does not waste a week: the **402 ms GPU segment is the floor, not a target** — BiRefNet always infers at 1024^2 whatever you hand it, so shrinking the input gains nothing and feeding it the full upload is slower *and* visibly worse. And **the 375 ms gate is not fat** — blur detection needs real pixels; downscaling first destroys the signal it measures.

The one thing still slow and not on that list: `ai/service.py` has **no startup warm**. `segmenter.warm()` exists and nothing calls it, so the model loads lazily on the first `/enhance` — **3486 ms cold load, then 899 ms for the first inference** against 402 ms warm. Every service restart re-arms this and **it lands on the first photograph of a demo**. A lifespan hook plus one dummy inference removes ~4 s and changes nothing else. It is a ten-line fix that is not done.

Q2.6 — "One GPU, one worker thread. What happens when a thousand artisans upload at once?"

Think. This is the scalability question in the PS, aimed at the narrowest part of our system. Give the arithmetic, not reassurance.

Today, and it is a deliberate choice rather than an oversight. `ai/enhance/jobs.py`, in its own docstring:

"One worker thread, deliberately. There is one GPU and BiRefNet holds ~1.6 GB of it while running. Two concurrent jobs on a 4 GB card is an out-of-memory crash, not double the throughput, so jobs queue and run one at a time."

The arithmetic. 1.3 s per photograph ⇒ **~46 photographs/minute, ~2,700/hour** per GPU at full occupancy. A thousand simultaneous uploads means the last artisan in the queue waits **~21 minutes**. They will not wait. Nothing in the app tells them their position.

Two further limits, both real, both written down in the code:

1. **The job table is process-local.** An `OrderedDict` in one process, bounded at 512 remembered jobs. **Restart the service and in-flight jobs are lost and their ids stop resolving.** It does not survive a second replica

either — a poll routed to the other process finds nothing.

2. **GET /api/enhanced/{path}** is demo scaffolding and must be deleted before production. It serves the AI box's local disk, and the URLs it builds embed whichever host the caller used — on a phone hotspot that is a DHCP lease that moves within the hour. `_record_variants` writes those URLs to the database, **so rows published this way go stale when the laptop's address changes.**

At scale — and the swap surface is deliberately two functions. `submit()` and `get()` are the whole interface a queue has to replace (`docs/decisions.md` #2 settled Redis + RQ). Then:

- N worker processes, one per GPU, each holding one BiRefNet instance
- Autoscale on **queue depth**, not CPU — the queue is the honest signal
- **Backpressure that speaks:** at depth > N, the app says *"thodi der lagegi"* with a position, instead of a spinner. Rule 3 again
- **BiRefNet-lite is a live capacity lever:** 273 ms and 851 MiB against 645 ms and 1615 MiB. It is **~2.4× the throughput and half the VRAM** — two lite workers fit where one full one did. Already benchmarked, already "held in reserve"
- **The enhance queue tolerates preemption** — it is asynchronous by contract, so spot/preemptible GPU instances are usable, which is 60–70% off the compute line

Capacity in one sentence, for the costing conversation: one GPU at 50% utilisation handles ~1.3 million photographs a month. At ten photographs per artisan per month that is **~130,000 artisans per GPU**. The GPU is not what limits this system. See the costing document.

Breaks when. Today: a service restart during a demo, or two replicas behind a load balancer. Both are fixed by the same two-function swap, and neither is fixed yet.

Q2.7 — "You are enhancing photographs of products people will buy. Where is the line between enhancement and lying?"

Think. This is an ethics question with a technical answer, and we have an unusually strong one. Lead with the rule, then the mechanism that makes the rule enforceable rather than aspirational.

The rule (`CLAUDE.md`, non-negotiable rule 1, and `docs/Master-Technical-Reference.md` §5.6):

| *Never change colour or shape. Enhance, don't misrepresent.*

Explicitly rejected, by name, in our own repo: super-resolution, generative backgrounds, diffusion relighting, and saturation boost beyond +10%. Rejected as **fabrication**, not as scope.

Two reasons, one practical and one legal. Return rate destroys artisan income. And misrepresentation is a listing violation that gets the product pulled — on the platform we headlined.

The mechanism that turns the rule into a guarantee. For the opt-in generative secondary images, we copy the one architectural choice PhotoRoom made (`docs/Application-Architecture.md` §7.2):

| Outpaint, never inpaint. Then composite the original subject pixels back on top, verbatim.

Mask boundaries are rigid; the model cannot regenerate anything inside the mask. **Even if the generative model hallucinates wildly, the product region is bit-identical to what the artisan photographed.** That is a mathematical guarantee rather than a hope, and it is the difference between a claim we can put on a slide unhedged and one we cannot.

Three more guards:

- **Generated images are always secondary. Never the main image.** A generated model shot as the primary is exactly the "inaccurate representation" that pulls a listing.
- **colour_confirmed gates publish.** Rule 4. Nothing goes out until the person holding the object says the colour is true.
- **Never destroy the original.** Rule 2: *store a recipe, render on demand*. The original is always recoverable, so no enhancement is irreversible, and any dispute can be settled against the artisan's own file.

One deliberate product decision that follows from this: an enhancement-intensity slider. Handicraft imperfection is a *selling point* — over-processed photographs look factory-made, which is the opposite of what the buyer is paying for.

Breaks when. The contact shadow on secondary images is ~20 lines of OpenCV — blur the mask, skew by the light angle, multiply. It is geometrically plausible, not physically correct. On a phone screen nobody can tell; under scrutiny it is a synthetic shadow, and it is on a secondary image where we have said generation may occur.